

METRIC_DEPENDENCY_GRAPH_PLAIN

Helios Metric Dependency Graph — In Plain English

Plain-language companion to `docs/architecture/METRIC_DEPENDENCY_GRAPH.md` . For the exact spec, read that / its PDF.

In one sentence

This is the family tree of Helios's metrics: it shows which numbers are built from which other numbers, so when something moves you can trace *why* — and when something breaks you can see what else breaks with it.

Why this matters to you

When `revenue` drops, the AI doesn't guess — it walks down a fixed map: was it fewer visits, a worse funnel step, or smaller baskets? That map is this document. It also tells you the "blast radius": if one raw count is wrong, which downstream metrics go wrong too. Knowing the tree is how you debug fast and how the diagnosis stays trustworthy instead of hand-wavy.

The big ideas, simply

1. Three layers, and arrows only point down. Every metric sits in one of three levels:

- **Leaves (Layer 0)** — the raw building blocks. Simple `count` or `sum` over a real column (e.g. `sessions`, `revenue`, `gross_revenue`, `purchasing_sessions`). These are the *only* things that touch physical data.
- **Ratios (Layer 1)** — exactly one number divided by another (e.g. `aov` = `gross_revenue` / `orders`).
- **Derived (Layer 2)** — formulas over the metrics below them (e.g. `revenue_per_session` = `revenue` / `sessions`).

No metric depends on something above it, so there are no loops. Clean and traceable.

2. Rates are always recomputed, never stored. Every rate is `SUM(numerator)/SUM(denominator)` after grouping. Averaging pre-divided ratios would produce Simpson's-paradox lies, so the system refuses to.

3. Some leaves are high-leverage. A bug in `sessions` poisons five-plus downstream metrics (`engagement_rate`, `session_conversion_rate`, `channel_conversion_rate`, `revenue_per_session`, `traffic_share`). Same for `purchasing_sessions`. These are the leaves to test hardest, because their blast radius is huge.

How it fits together

The metrics cluster into families:

Family	Example metrics
Revenue	<code>revenue</code> , <code>gross_revenue</code> , <code>aov</code> , <code>revenue_per_session</code> , <code>revenue_per_user</code>
Traffic	<code>sessions</code> , <code>users</code> , <code>engaged_sessions</code> , <code>engagement_rate</code>
Funnel	<code>view_to_cart_rate</code> , <code>cart_to_checkout_rate</code> , <code>checkout_to_purchase_rate</code> , <code>session_conversion_rate</code>
Retention	<code>day_1_retention</code> , <code>day_7_retention</code> , <code>day_30_retention</code> , <code>repeat_purchase_rate</code>
Acquisition	<code>channel_revenue</code> , <code>channel_conversion_rate</code> , <code>cac_proxy</code> , <code>traffic_share</code>
Product	<code>product_revenue</code> , <code>product_attach_rate</code> , <code>product_view_rate</code>

The funnel counts are **monotonic** (each step is smaller than the last), so step rates always land between 0 and 1.

A few cross-family links are intentional: e.g. `product_attach_rate` divides an item-level count by `orders` (an order-level metric). The system reconciles across those grains on purpose.

The identities — the algebra that makes diagnosis work

These exact equalities are the backbone. They let the AI attribute a dollar move to a specific cause:

- **`revenue_per_session` = `session_conversion_rate` × `aov`** — was it conversion or basket size?
- **`revenue` = `orders` × `aov`** — was it order volume or basket size?

- **session_conversion_rate** = **view_to_cart_rate** × **cart_to_checkout_rate** × **checkout_to_purchase_rate** (times the sessions→view_item entry step) — which funnel step regressed?

Chained together, any **revenue** anomaly walks down to exactly one of: **traffic volume** (**sessions**), a **specific funnel step**, or **basket size** (**aov**).

How a diagnosis actually runs

1. **Detect** the anomaly (Monitor).
2. **Decompose** the move with **decompose_change** into **mix_effect** + **rate_effect** + **interaction**.
3. **Read it:**
 - **mix-dominant** = the *composition* of traffic shifted (more low-converting channels) → it's an acquisition/allocation story, not a broken funnel.
 - **rate-dominant** = real in-segment behavior changed → fix the funnel/UX/pricing.
4. **Drill** the segment with the biggest *rate* effect, following the identities above.

traffic_share is the "mix weight" that tells you whether a move was just composition (Simpson's paradox) rather than a genuine change.

Easy things to get wrong

- **cac_proxy** is **NOT real CAC**. There's no cost data in this dataset. It's paid-sessions-per-new-customer, a volume efficiency proxy — never read it as dollars of acquisition cost.
- **Don't confuse mix and rate**. A "rate" finding can secretly be a mix shift hiding at a finer grain. Always re-split one level deeper before trusting it.
- **Retention denominators are fixed**. Always the original cohort size, never the survivors — that's why each later retention number is smaller.
- **Product view-to-purchase isn't a real within-session rate** — views and purchases can be different sessions/cookies. Treat it as an approximation.
- **Reconcile the decomposition**. **mix + rate + interaction** must equal the actual change within tolerance, or the finding is rejected.

Glossary — the exact words, demystified

- **leaf** — a base count/sum over a real column; the only metrics touching physical data.
- **ratio** — one metric divided by another (numerator / denominator).

- **derived** — a formula over other metrics.
- **blast radius** — everything downstream that breaks if a metric is wrong.
- **mix_effect / rate_effect / interaction** — the three pieces a change splits into: composition shift, in-segment behavior shift, and the overlap.
- **traffic_share** — a segment's share of sessions; the mix weight used to tell composition shifts from real changes.
- **identity** — an exact algebraic equality (like the three above) that always holds, used to attribute movement.
- **supporting metric** [S] — a helper count defined inside one cluster, not a headline number.

When to open the real doc

Open [pdf/docs/architecture/METRIC_DEPENDENCY_GRAPH.pdf](#) for the full per-metric dependency trees, the exact drill orders per headline metric, the complete identity derivations, and the consumed-by table mapping metrics to agents.